

Tema 11

Introducción a LINQ



Índice

- Introducción
- Filtrado
- Ordenación
- Selección
- Agregación
- Cuantificación



¿Qué es LINQ?

- **LINQ** (Language Integrated Query) es una característica de C# que permite realizar **consultas de datos** directamente en el lenguaje de programación, sin necesidad de escribir SQL ni interactuar directamente con bases de datos.
- LINQ se puede usar para consultar **distintos tipos de colecciones**, como arrays, listas, colecciones de objetos, bases de datos, archivos XML y mucho más.

1. Introducción



- LINQ permite expresar consultas en un estilo **declarativo**, especificando "qué" quieres hacer con los datos en lugar de "cómo" procesarlos (paradigma de la programación declarativa).
- Gracias a esto, LINQ mejora la legibilidad del código y reduce la cantidad de líneas necesarias para manipular colecciones de datos.

1. Introducción

Ventajas de usar LINQ

- **Legibilidad:** LINQ permite que las consultas sean más fáciles de leer y entender al mantener una sintaxis coherente.
- **Productividad:** Facilita la manipulación de datos en diversas estructuras (como listas y bases de datos) sin aprender nuevos lenguajes.

1. Introducción

- **Control de errores:** LINQ detecta errores en tiempo de compilación, aumentando la seguridad y evitando errores típicos de ejecución.
- **Eficiencia:** LINQ optimiza consultas internamente, lo que puede resultar en un rendimiento mejorado en comparación con otros métodos de iteración.

1. Introducción

¿Cómo funciona LINQ?

- En LINQ, las consultas se pueden escribir de dos maneras: **sintaxis de consulta** (Query Syntax) y **sintaxis de métodos** (Method Syntax).
- Ambas tienen sus ventajas, y la elección entre ellas depende del estilo y la complejidad de la consulta que se esté realizando.

1. Introducción

- **Query Syntax:** Es similar al estilo de SQL y permite escribir consultas en un formato declarativo.
- **Method Syntax:** Utiliza métodos de extensión sobre colecciones para realizar las consultas y transformaciones en los datos.

1. Introducción

Ejemplo

- En este ejemplo, `queryNumbers` utiliza `from`, `where`, y `select`, similar a SQL, mientras que `methodNumbers` usa el método `Where( )` para filtrar la lista de números.

1. Introducción

```
1 List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6,
2
3 // Query Syntax
4 IEnumerable<int> queryNumbers = from num in numbers
5                                 where num % 2 == 0
6                                 select num;
7
8 // Method Syntax
9 var methodNumbers = numbers.Where(num => num % 2 == 0);
```

- Ambas sintaxis son equivalentes y producen el mismo resultado.
- La *method syntax* proporciona un estilo es más conciso y fácil de usar cuando se necesita aplicar funciones más complejas.
- La *query syntax* facilita algunas operaciones como las uniones. Además, si estás acostumbrado a SQL, su sintaxis es similar.

1. Introducción

Componentes de una consulta LINQ

- Una consulta LINQ generalmente sigue estos pasos básicos:
 - Origen de los datos
 - Creación de la consulta
 - Ejecución de la consulta

1. Introducción

- **Origen de los datos:** La colección o estructura de datos sobre la que se realizará la consulta.
- **Creación de la consulta:** La lógica que define el filtro, orden, transformación, etc., de los datos.
- **Ejecución de la consulta:** La consulta se ejecuta al iterar sobre ella o convertirla a otro tipo, como una lista (`ToList()`), ya que LINQ utiliza "ejecución diferida" (*deferred execution*).

1. Introducción

Ejecución Diferida (Deferred Execution)

- En LINQ, el filtrado y la mayoría de las operaciones sobre colecciones no se ejecutan inmediatamente.
- Esto se llama **ejecución diferida** (deferred execution).
- La consulta se "prepara" pero solo se ejecuta cuando realmente se necesita el resultado, como al iterar sobre la colección filtrada o al usar métodos como `ToList()`.

1. Introducción

- El filtrado es una de las operaciones más comunes y potentes que se pueden realizar con LINQ.
- Permite seleccionar elementos de una colección que cumplan con ciertas condiciones, utilizando el operador `Where`.
- El filtrado es útil para reducir el conjunto de datos en función de criterios específicos y seleccionar solo aquellos elementos que cumplen con esos criterios.

2. Filtrado

Ejemplo

- En este ejemplo, realizamos 2 consultas sobre una lista de enteros. La primera, en *query syntax*, contiene los valores (48, 280, 48, 120, 0). La segunda, en *method syntax*, contiene los valores (280, 120).

2. Filtrado

```
1 List<int> numbers = new List<int>() { 3, 48, -13, 280, 4  
2  
3 IEnumerable<int> evenNumbersQuery = from num in numbers  
4                                     where num % 2 == 0  
5                                     select num;  
6  
7 List<int> bigNumbersMethod = numbers.Where(n => n > 50).
```

- También es posible combinar varias condiciones en un filtro para obtener resultados más específicos.
- Vamos a suponer una clase Person de la que vamos a tener una lista y queremos seleccionar aquellas que sean mayores de 18 años y cuyo nombre comience con "A".

2. Filtrado

```
1 class Person
2 {
3     public string Name { get; set; }
4     public int Age { get; set; }
5 }
```

Ejemplo

2. Filtrado

```
1  List<Person> people = new List<Person>
2  {
3      new Person { Name = "Antonio", Age = 22 },
4      new Person { Name = "Ana", Age = 8 },
5      new Person { Name = "Luis", Age = 15 },
6      new Person { Name = "Alfredo", Age = 30 },
7      new Person { Name = "Beatriz", Age = 28 },
8  };
9
10 IEnumerable<Person> filteredPeopleQuery = from p in people
11                                             where p.Age > 18 && p.Name.StartsWith("A")
12                                             select p;
13
14 List<Person> filteredPeopleMethod = people
15     .Where(p => p.Age > 18 && p.Name.StartsWith("A"))
16     .ToList();
17
18 foreach (Person person in filteredPeopleQuery)
19 {
20     Console.WriteLine("{0} ({1})", person.Name, person.Age);
21 }
22
```

- En el ejemplo anterior, realizamos el filtrado tanto en *query* como en *method syntax*.
- Para realizar filtros combinados, utilizamos el operador `&&` en los `Where`.
- De esta manera, podemos crear consultas complejas con diferentes condiciones para lograr filtrados (y otras operaciones que veremos después) realmente potentes con una sintaxis sencilla.

2. Filtrado

Buenas Prácticas para Filtrado en LINQ

- **Usar expresiones claras:** Las expresiones en el filtro deben ser fáciles de entender para mantener el código legible.
- **Evitar cálculos innecesarios:** Coloca las condiciones de filtro más restrictivas primero para mejorar el rendimiento.

2. Filtrado

- **Usar First, FirstOrDefault, Single, y SingleOrDefault cuando corresponda:** Si solo se espera un resultado, estos métodos pueden simplificar el código y hacerlo más eficiente.
- Veremos estos operadores en detalle en el punto 7, sobre operaciones condicionales.

2. Filtrado

Vídeo práctico

2. Filtrado

Vídeo práctico - código

2. Filtrado

```
1  public class Product
2  {
3      public string Name { get; set; }
4      public double Price { get; set; }
5      public int Stock { get; set; }
6      public bool IsAvailable { get; set; }
7  }
8
9  public class Program
10 {
11     static void Main(string[] args)
12     {
13         List<int> numbers = new List<int>() { 1, 0, -5,
14
15         IEnumerable<int> positiveNumbers = from n in numbers
16                                             where n >= 0
17                                             select n;
18
19         Console.WriteLine("Números positivos: ");
20         foreach (int n in positiveNumbers)
21         {
22             Console.Write(n + " ");
23         }
24     }
25 }
```

- La ordenación en LINQ permite organizar los elementos de una colección en un orden específico, ya sea ascendente o descendente.
- LINQ proporciona los operadores `OrderBy`, `OrderByDescending`, `ThenBy`, y `ThenByDescending` para este fin.
- Estas operaciones son útiles para organizar datos antes de mostrarlos o para realizar búsquedas más eficientes.

3. Ordenación

Operador OrderBy

- El operador OrderBy en LINQ permite **ordenar** los elementos de una colección en orden ascendente según un criterio especificado.
- Es particularmente útil en aplicaciones donde los datos necesitan presentarse o procesarse en un orden específico, como en la visualización de listas ordenadas o al trabajar con jerarquías.

3. Ordenación

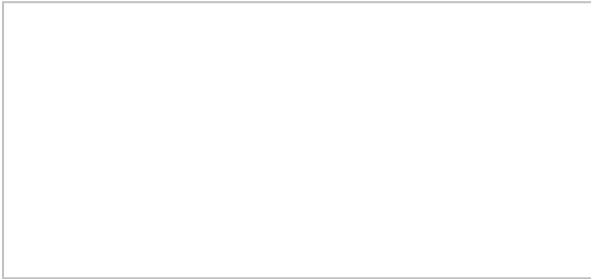
Ejemplo

3. Ordenación

```
1  class Product
2  {
3      public string Name { get; set; }
4      public double Price { get; set; }
5  }
6
7  List<Product> products = new List<Product>
8  {
9      new Product { Name = "Portátil MSI", Price = 1300 },
10     new Product { Name = "Ratón Logitech", Price = 44.99 },
11     new Product { Name = "PlayStation 5", Price = 799.99 },
12     new Product { Name = "iPhone 16", Price = 879 },
13     new Product { Name = "Auriculares Razer", Price = 44.99 }
14 };
15
16 // Query Syntax
17 IEnumerable<Product> sortedByPriceQuery = from p in products
18                                             orderby p.Price
19                                             select p;
20
21 // Method Syntax
```

- En el ejemplo anterior tenemos una lista de productos y queremos ordenarla en base al precio en orden ascendente.
- En este caso, ambas sintaxis devuelven una lista de productos ordenados por precio, desde el más barato al más caro.

3. Ordenación



Operador OrderByDescending

- Si queremos ordenar los elementos en orden descendente, debemos usar `OrderByDescending`.
- Este operador es una variación de `OrderBy` que invierte el orden, colocando primero el elemento con el valor más alto.
- En *query syntax*, ponemos `descending` después del criterio de ordenación (ej: `orderby p.Price descending`)

3. Ordenación

- Por ejemplo, si queremos ordenar por precio descendente el ejemplo anterior:

3. Ordenación

```
1 // Query Syntax
2 IEnumerable<Product> sortedByPriceDescQuery = from p in products
3                                                  orderby p.Price descendi
4                                                  select p;
5
6 // Method Syntax
7 List<Product> sortedByPriceDescMethod = products.OrderByDescending(p =
8
9 foreach(Product product in sortedByPriceDescMethod)
10 {
11     Console.WriteLine("{0} ({1})", product.Name, product.Price);
```

Ordenación compuesta

- En ocasiones, es útil ordenar una lista por más de un criterio.
- Por ejemplo, si varios productos tienen el mismo precio, podríamos querer ordenarlos alfabéticamente por nombre.
- En este caso, la *query syntax* y la *method syntax* difieren ligeramente.

3. Ordenación

- Si utilizamos *query syntax*, podemos especificar varios criterios de ordenación separados por coma (,) en el mismo orderby o también podemos encadenar diferentes operadores orderby.
- En el caso de *method syntax*, debemos utilizar los operadores ThenBy o ThenByDescending después del primer OrderBy.

3. Ordenación

Ejemplo

3. Ordenación

```
1 // Query Syntax
2 IEnumerable<Product> compoundSortedQuery = from p in products
3                                             orderby p.Price desc
4                                             select p;
5
6 // Method Syntax
7 List<Product> compoundSortedMethod = products.OrderByDescending
8
9 foreach (Product product in compoundSortedQuery)
10 {
11     Console.WriteLine("{0} ({1})", product.Name, product.Price)
12 }
```

- En el ejemplo anterior, realizamos en primer lugar una ordenación por precio (ascendente).
- Para los elementos que tienen el mismo valor, volvemos a realizar una ordenación, esta vez por nombre.
- Ambas sintaxis ofrecen el mismo resultado, pero debemos utilizar operadores diferentes en este caso.

3. Ordenación

Ordenación de datos nulos

- En casos donde algunas propiedades pueden ser `null`, es importante gestionar correctamente la ordenación para evitar excepciones o comportamientos indeseados.
- `OrderBy` maneja los valores `null` al principio de la lista en orden ascendente y al final en orden descendente (`null` tiene el **valor mínimo** al ordenar).

3. Ordenación

- Si queremos cambiar este comportamiento, se puede realizar una ordenación condicional.
- Dentro de los `bool`, `OrderBy` ordena en primer lugar los `false` y después los `true`.
- Así pues, si ordenamos en primer lugar igualando a `null`, nos aparecerán en primer lugar los valores que no sean `null` (condición a `false`) y después los `null`.
- De esta manera colocamos los `null` al final.

3. Ordenación

Ejemplo

3. Ordenación

```
1  class Customer
2  {
3      public string Name { get; set; }
4      public string Address { get; set; }
5  }
6
7  List<Customer> customers = new List<Customer>
8  {
9      new Customer { Name = "Manuel", Address = "C/ Rosar:
10     new Customer { Name = "Antonio", Address = null },
11     new Customer { Name = "Marcial", Address = "C/ Mayor
12     new Customer { Name = "Luis", Address = null },
13 };
14
15 // Ordenar poniendo los clientes sin dirección al final
16 List<Customer> sortedCustomers = customers
17     .OrderBy(c => c.Address == null)
18     .ThenBy(c => c.Name)
19     .ToList();
20
21 foreach (Customer customer in sortedCustomers)
22 {
23     Console.WriteLine(customer.Name);
24 }
```

- En el ejemplo anterior ordenamos la lista de clientes por nombre.
- Pero antes hacemos el `OrderBy(c => c.Address == null)`.
- De esta manera, dejamos al final de la lista los valores nulos, ya que la expresión devuelve `true` y la ordenación de `bool` pone en primer lugar los `false` (en este caso, los que sí tienen dirección asignada).

3. Ordenación

Ordenación con Expresiones Complejas

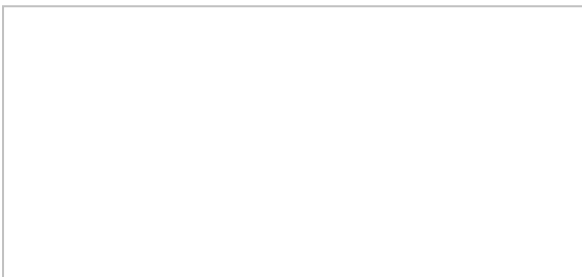
- OrderBy permite utilizar **cualquier expresión lógica** que retorne un valor comparable.
- Por ejemplo, si queremos ordenar en base al número de caracteres del nombre del producto:

3. Ordenación

```
1 List<Product> sortedByLengthOfNameMethod = product
2     .OrderBy(p => p.Name.Length)
3     .ToList();
4
5 foreach (Product product in sortedByLengthOfNameM
6 {
7     Console.WriteLine("{0} ({1})", product.Name, |
```

- En el ejemplo anterior, el criterio de ordenación no es Name, sino el valor de Length del Name de cada uno de los elementos.
- También podemos utilizar otras expresiones complejas como por ejemplo, ordenar por la posición de la primera letra 'a':

3. Ordenación



```
1 List<Product> sortedByFirstAMethod =  
2     .OrderBy(p => p.Name.IndexOf('a'))  
3     .ToList();
```

Buenas Prácticas en la Ordenación

- **Evitar ordenaciones innecesarias:** La ordenación puede afectar el rendimiento si se usa de forma indiscriminada en grandes colecciones.
- **Usar criterios específicos:** Si solo se necesita ordenar por un criterio, evitar la sobrecarga de `ThenBy` o `ThenByDescending`.

3. Ordenación

- **Ordenar Antes de Filtrar o Seleccionar:** A menudo, es eficiente ordenar primero la colección antes de aplicar otros operadores de consulta como `Where` o `Select`, dependiendo del contexto del uso.
- **Considerar el Tipo de Datos:** Utilizar `OrderBy` en tipos de datos comparables, como enteros, cadenas o fechas, garantiza una ordenación predecible.

3. Ordenación

Vídeo práctico

3. Ordenación

Vídeo práctico - código

3. Ordenación

```
1  public class Product
2  {
3      public string Name { get; set; }
4      public double Price { get; set; }
5      public int Stock { get; set; }
6      public bool IsAvailable { get; set; }
7  }
8
9  public class Program
10 {
11     static void Main(string[] args)
12     {
13         List<int> numbers = new List<int>() { 1, 0, -5, 27,
14
15         List<Product> products = new List<Product>()
16         {
17             new Product() { Name = "Impresora", Price = 89.9,
18             new Product() { Name = "Impresora", Price = 117,
19             new Product() { Name = null, Price = 45, Stock = 10,
20             new Product() { Name = "Micrófono", Price = 35,
21             new Product() { Name = "Portátil", Price = 1150,
22             new Product() { Name = "Portátil", Price = 500,
```

- El operador `Select` en LINQ permite proyectar, o seleccionar, **valores específicos** de una colección, transformándolos y creando una nueva colección basada en la original.
- Es particularmente útil para extraer solo la información relevante de un conjunto de datos o aplicar alguna transformación a sus elementos.

4. Selección

Uso de Select

- La operación `Select` se usa para tomar cada elemento de una colección y transformarlo o extraer una parte específica.
- El resultado es una nueva colección que contiene únicamente la información seleccionada.

4. Selección

Ejemplo

4. Selección

```

1  class Person
2  {
3      public string Name { get; set; }
4      public int Age { get; set; }
5      public string Id { get; set; }
6  }
7
8  List<Person> people = new List<Person>
9  {
10     new Person { Name = "Alicia", Age = 25, Id = "58"
11     new Person { Name = "Patricia", Age = 8, Id = "4"
12     new Person { Name = "Jorge", Age = 35, Id = "094"
13     new Person { Name = "Joaquín", Age = 35, Id = "3"
14 };
15
16 // Seleccionamos solo el nombre de cada persona
17 List<string> names = people.Select(p => p.Name).ToLi

```

- En el ejemplo anterior, tenemos una lista de `Person`, clase que tiene las propiedades de nombre (`string`), edad (`int`) y DNI (`string`).
- Hacemos un `Select` para quedarnos únicamente con el nombre. El resultado es una lista de `string` que contiene la propiedad `Name` de cada uno de los elementos de la colección de `Person`.

4. Selección

Selección de Propiedades Complejas

- Select también permite seleccionar más de una propiedad de cada elemento.
- Podemos devolver un **nuevo objeto anónimo** o un tipo específico con los valores deseados.

4. Selección

Ejemplo

- En este ejemplo, seleccionamos el nombre y la edad, creando una lista de objetos anónimos que contienen las propiedades seleccionadas.
- Debemos utilizar la palabra clave `var`, que vimos en el tema 2, para declarar la variable.

4. Selección

```
1 var nameAndAge = people.Select(p => new { p.Name, p.Age })
```

Transformaciones de Datos

- El operador `Select` permite aplicar transformaciones a los datos.
- Podemos usar `Select` para realizar operaciones como cambiar mayúsculas y minúsculas, calcular valores basados en propiedades existentes, o crear representaciones personalizadas de los datos.

4. Selección

Ejemplo

- En este ejemplo, creamos una lista de `string` que contiene los nombres transformados a mayúsculas.

4. Selección

```
1 List<string> upperCaseNames = people.Select(p => p.Name.ToU|
2
3 foreach(string name in upperCaseNames)
4 {
5     Console.WriteLine(name);
6 }
```



ÍNDICES CON **SELECT**

- **Se**lect admite una sobrecarga que permite acceder al índice de cada elemento en la colección.
- Esto puede ser útil para situaciones en las que el índice es necesario para realizar alguna transformación adicional.

4. Selección

Ejemplo

- En este ejemplo, creamos una lista de objetos anónimos utilizando el índice de la iteración sobre la colección.

4. Selección

```
1 var indexedNames = people.Select((n, index) => new { Index = i
```

- También podemos utilizar el índice en cadenas de texto o realizar operaciones con él:
- En este caso, el resultado es una lista de cadenas con formato personalizado en el que además utilizamos el índice de la iteración.

4. Selección

```
1 List<string> indexedNames = people
2     .Select((p, index) => string.Format("{0}. {1}", index + 1,
3
4 foreach (string name in indexedNames)
5 {
6     Console.WriteLine(name);
```



Buenas Prácticas con Select

- **Evitar Proyecciones Innecesarias:** Si solo se necesita cierta información de la colección, es recomendable seleccionar únicamente las propiedades necesarias para reducir el uso de memoria.
- **Mantener el Código Claro:** Al usar Select, es útil ser específico en cuanto al resultado deseado, evitando expresiones complejas que reduzcan la legibilidad.

4. Selección

Vídeo práctico

4. Selección

Vídeo práctico - código

4. Selección

```
1  public class Product
2  {
3      public string Name { get; set; }
4      public double Price { get; set; }
5      public int Stock { get; set; }
6      public bool IsAvailable { get; set; }
7  }
8
9  public class Program
10 {
11     static void Main(string[] args)
12     {
13         List<Product> products = new List<Product>()
14         {
15             new Product() { Name = "Impresora", Price = 150, Stock = 10, IsAvailable = true },
16             new Product() { Name = "Impresora", Price = 150, Stock = 10, IsAvailable = true },
17             new Product() { Name = "Micrófono", Price = 50, Stock = 5, IsAvailable = true },
18             new Product() { Name = "Portátil", Price = 450, Stock = 3, IsAvailable = true },
19             new Product() { Name = "Portátil", Price = 450, Stock = 3, IsAvailable = true },
20             new Product() { Name = "Portátil", Price = 450, Stock = 3, IsAvailable = true },
21             new Product() { Name = "Monitor", Price = 100, Stock = 2, IsAvailable = true },
22             new Product() { Name = "Tarjeta gráfica", Price = 120, Stock = 1, IsAvailable = true },
23         }
24     }
25 }
```

- La agregación en LINQ permite realizar **operaciones acumulativas** en una colección de datos, tales como sumar valores, calcular promedios, encontrar valores máximos o mínimos, entre otros.
- Estas operaciones ayudan a obtener estadísticas y resultados acumulados a partir de datos.
- LINQ proporciona métodos de extensión específicos para realizar estas operaciones de manera eficiente y con una sintaxis clara.

5. Agregación

Sum

- La operación Sum calcula la suma total de una secuencia numérica y lo devuelve en un valor del mismo tipo numérico.
- En este ejemplo, para la lista de Person que utilizamos en el ejemplo del punto anterior, totalAge contendría 103, lo que representa la suma de todas las edades.

5. Agregación

```
1 int totalAge = people.Sum(p => p.Age);
```

Average

- La operación `Average` devuelve un `double` con el valor promedio de una secuencia numérica.
- Para el ejemplo con el que estamos trabajando, `averageAge` almacena el valor 25'75.

5. Agregación

```
1 double averageAge = people.Average()
```

Min y Max

- Las operaciones Min y Max devuelven un valor numérico con el mínimo y el máximo de una secuencia numérica, respectivamente.
- En nuestro ejemplo, minAge contiene el valor 8 y maxAge el valor 35.

5. Agregación

```
1 int minAge = people.Min(p => p.Age);  
2 int maxAge = people.Max(p => p.Age);
```

Count

- El operador Count devuelve un `int` con el número de elementos de una colección.
- Podemos añadir condiciones al operador para que cuente únicamente los elementos que la cumplen.
- En nuestro ejemplo, `totalPeople` almacena el valor 4 y `adultPeople` el valor 3.

5. Agregación

```
1 int totalPeople = people.Count();
```

Aggregate

- El método `Aggregate` permite realizar una agregación personalizada sobre los elementos de una colección.
- A diferencia de los métodos anteriores, `Aggregate` acepta una función de acumulación en la que puedes definir tu propia lógica de agregación.
- Es útil para casos en los que deseas realizar una acumulación específica.

5. Agregación

- Esto significa que el método Aggregate realiza una operación sobre cada elemento de una lista que tiene en cuenta el resultado de las operaciones anteriores.
- Necesita operar sobre 2 elementos sobre los que realizará la operación. El primer paso será entre los 2 primeros, el segundo entre el resultado anterior y el tercer elemento y así sucesivamente.
- También podemos especificar un valor inicial para realizar la agregación con el primer elemento de la colección (en vez de los 2 primeros elementos).

5. Agregación

Ejemplo

- En este ejemplo, la variable `sum` almacena 26 después de la operación.
- En el primer paso suma $12 + 2$. Después 14 (resultado de la operación anterior) $+ 8$. Por último $22 + 4$ y finaliza la agregación de todos los elementos.

5. Agregación

```
1 var numbers = new[] { 12, 2, 8, 4 }  
2 var sum = numbers.Aggregate(0, (a, b)
```

Ejemplo

- En este ejemplo, realizamos la agregación sobre una lista de `string` y además especificamos un valor inicial.
- Agrega cada par elementos separándolos con una coma y espacio.
- Al finalizar la operación, `resultString` contiene:

5. Agregación

```
1 string[] namesArray = ["Antonio", "Paco", "Luis", "Ana"];
```

"José, Antonio, Paco, Luis, Ana"

Buenas Prácticas para Operaciones de Agregación

- **Evitar el uso innecesario de Aggregate:** Si existen métodos específicos como Sum o Average, úsalos, ya que son más directos y claros.
- **Filtrar antes de Agregar:** Aplica filtros (Where) antes de realizar la agregación para evitar cálculos innecesarios.

5. Agregación

- **Optar por Ordenamiento Condicional:** Al buscar valores máximo o mínimo, utilizar directamente Max o Min suele ser más eficiente que OrderBy o OrderByDescending y elegir el primero.

5. Agregación

Vídeo práctico

5. Agregación

Vídeo práctico - código

5. Agregación

```
1  public class Product
2  {
3      public string Name { get; set; }
4      public double Price { get; set; }
5      public int Stock { get; set; }
6      public bool IsAvailable { get; set; }
7  }
8
9  public class Program
10 {
11     static void Main(string[] args)
12     {
13         List<Product> products = new List<Product>()
14         {
15             new Product() { Name = "Impresora", Price = 89
16             new Product() { Name = "Impresora", Price = 11
17             new Product() { Name = "Micrófono", Price = 35
18             new Product() { Name = "Portátil", Price = 115
19             new Product() { Name = "Portátil", Price = 599
20             new Product() { Name = "Portátil", Price = 850
21             new Product() { Name = "Monitor", Price = 125,
22             new Product() { Name = "Tarjeta gráfica", Pric
```

- La cuantificación en LINQ se refiere a la evaluación de condiciones sobre los elementos de una colección para determinar si cumplen ciertos criterios.
- Es útil para realizar verificaciones rápidas en colecciones, tales como ver si todos los elementos cumplen una condición o si al menos uno de ellos lo hace.

6. Cuantificación

- LINQ ofrece tres métodos de cuantificación principales:
- **All**: Evalúa si todos los elementos de la colección cumplen con una condición especificada.
- **Any**: Evalúa si al menos uno de los elementos de la colección cumple con una condición.
- **Contains**: Evalúa si al menos uno de los elementos de la colección es igual a la condición.

6. Cuantificación

Método All

- El método All comprueba si **todos los elementos** de una colección cumplen una determinada condición.
- Este método devuelve un valor `true` si todos los elementos cumplen con la condición; de lo contrario, devuelve `false`.

6. Cuantificación

Ejemplo

- En este ejemplo, comprobamos si todos los elementos de una lista de enteros son positivos. En este caso, `allPositive` se evaluará como `true` ya que no hay ningún número negativo en la lista.

6. Cuantificación

```
1 List<int> numbers = new List<int> { 91, 2, 4,  
2 bool allPositive = numbers.All(n => n > 0);
```

Método Any

- El método Any permite verificar si existe **al menos un elemento** en una colección que cumpla con una condición.
- Si al menos un elemento cumple la condición, devuelve `true`; si ninguno lo cumple, devuelve `false`.

6. Cuantificación

Ejemplo

- En este ejemplo, comprobamos si algún número de la lista es negativo. Como el elemento '-4' cumple la condición, `anyNegative` se evalúa a `true`.

6. Cuantificación

```
1 List<int> numbers = new List<int> { 91, 2, -4, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100 };
2 bool anyNegative = numbers.Any(n => n < 0);
```

Método Contains

- El método Contains lo vimos en el tema de las colecciones y permite verificar si al menos un elemento es igual a otro pasado como condición.
- En este caso, al contrario que Any, no permite expresiones personalizadas, por lo que únicamente compara con un valor del mismo tipo que la colección.

6. Cuantificación

Ejemplo

- En este ejemplo, comprobamos si algún número de la lista es exactamente 5. Como el número 5 se encuentra en la lista, `contains5` se evalúa como `true`.

6. Cuantificación

```
1 List<int> numbers = new List<int> { 91, 2, -4,  
2 bool contains5 = numbers.Contains(5);
```

Buenas Prácticas en Cuantificación

- **Evitar evaluaciones innecesarias:** Si solo necesitas verificar una condición sobre un subconjunto específico, usa métodos de filtrado como `Where` antes de aplicar `All` o `Any`.
- **Comprender las limitaciones de `All` y `Any`:** Estos métodos se utilizan para verificar condiciones booleanas sobre la colección completa; si necesitas realizar evaluaciones más complejas, es posible que debas combinar `All` y `Any` con otros métodos LINQ.

6. Cuantificación

- **Simplicidad:** Los métodos `All` y `Any` ofrecen una forma clara y concisa de verificar condiciones en colecciones. Úsalos en lugar de bucles `foreach` o `for` cuando solo necesitas evaluar una condición sobre los elementos de la colección.

6. Cuantificación

Vídeo práctico

6. Cuantificación

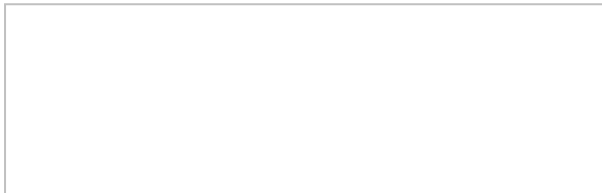
Vídeo práctico - código

6. Cuantificación

```
1  public class Product
2  {
3      public string Name { get; set; }
4      public double Price { get; set; }
5      public int Stock { get; set; }
6      public bool IsAvailable { get; set; }
7  }
8
9  public class Program
10 {
11     static void Main(string[] args)
12     {
13         List<Product> products = new List<Product>()
14         {
15             new Product() { Name = "Impresora", Price = 89,
16             new Product() { Name = "Impresora", Price = 115,
17             new Product() { Name = "Micrófono", Price = 20,
18             new Product() { Name = "Portátil", Price = 1150,
19             new Product() { Name = "Portátil", Price = 599,
20             new Product() { Name = "Portátil", Price = 850,
21             new Product() { Name = "Monitor", Price = 125,
22             new Product() { Name = "Tarjeta gráfica", Price = 150
```

Diapositiva final

En este tema hemos visto una introducción a LINQ, un lenguaje de consultas integrado en C# que permite operar sobre diferentes fuentes de datos. Hemos conocido algunas de las operaciones básicas y hemos visto la potencia que ofrece.



Speaker notes